Da rede neural à PINN

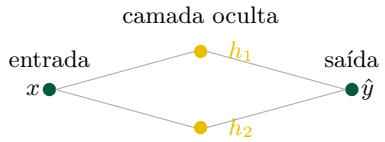
Revisão rápida para a implementação com DeepXDE

Tópicos Especiais em Transferência de Calor

Prof. MSc. Paulo de Souza Silva

1. Uma rede pequena: arquitetura 1–2–1

Considere uma rede com uma entrada x, dois neurônios na camada oculta e uma saída $\hat{y}$.



Para o primeiro neurônio oculto, calcula-se a função linear sobre x e executa a função de ativação σ:

$$z_1 = w_1 x + b_1, \qquad h_1 = \sigma(z_1),$$

A **função de ativação** σ é uma função de ativação genérica, isto é, pode representar uma Tangente Hiperbólica, uma ReLU, uma Sigmóide etc.

para o segundo neurônio na camada oculta, fazemos o mesmo processo:

$$z_2 = w_2 x + b_2, \qquad h_2 = \sigma(z_2).$$

Após a ativação nos neurônios temos como resultado os valores de h_1 e h_2, ambos irão compor a única saída da rede, o que segue:

$$\boxed{\hat{y} = v_1 h_1 + v_2 h_2 + b_3.}$$

Para uma variável contínua, como temperatura, normalmente deixamos a saída **sem** uma nova função de ativação.

Os parâmetros treináveis são

$$\theta = \{w_1, b_1, w_2, b_2, v_1, v_2, b_3\},$$

o que totaliza um total de 7 parâmetros a serem treinados.

Pergunta

Quantos parâmetros seriam necessários treinar se a rede tivesse a forma: [2] + [3] * 2 + [1]?
E se fosse do tipo [2] + [20] * 3 + [1]?

Respostas: 25 e 921.

Mesmo com **uma entrada** e **uma saída** por amostra, a rede pode ser aplicada a vários valores x_i. O mesmo conjunto de parâmetros é reutilizado em todas as amostras.

2. Da previsão à função de perda

Se o valor correto y for conhecido, podemos medir o erro da previsão por

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2.$$

O fator utilizado $1/2$ é uma estratégia conveniente apenas para deixar o resultado da derivada, que iremos executar, mais simples. Para N amostras, usamos a média:

$$\mathcal{L}(\theta) = \frac{1}{2N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2.$$

Para encontrar os melhores valores do conjunto de parâmetros θ podemos utilizar estratégias de gradiente (exemplos: gradiente descendente, método Adam, LBFGS). Vamos exemplificar com gradiente descendente, para atualizar o valor de w_1 por exemplo, devemos executar a seguinte operação:

$$\boxed{w_{1,\,\text{novo}} = w_{1,\,\text{antigo}} - \eta\frac{\partial \mathcal{L}}{\partial w_1}}.$$

em que η é o valor da **taxa de aprendizado** escolhido para o modelo.

Lembrando que no nosso exemplo existem 7 parâmetros, o mesmo procedimento deve ser realizado para todos os pesos e vieses.

O Adam ajusta uma taxa de aprendizagem efetiva para cada parâmetro. O L-BFGS utiliza informações aproximadas de curvatura para determinar a direção e o tamanho do passo de otimização.

Perceba que precisamos da derivada da **Loss** com relação ao parâmetro w_1 isto é: $\dfrac{\partial \mathcal{L}}{\partial w_1}$. Para isso usamos a estratégia de **retro-propagação** (*backpropagation*) que varre a rede de trás para frente aplicando a regra da cadeia para descobrir como cada parâmetro contribuiu para a perda. Por exemplo:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial w_1}.$$

esse mesmo formato de conta é aplicado a todos os parâmetros que devem ser otimizados.

$$\frac{\partial \mathcal{L}}{\partial b_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial h_1}\frac{\partial h_1}{\partial z_1}\frac{\partial z_1}{\partial b_1}.$$

$$\frac{\partial \mathcal{L}}{\partial v_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial v_1}$$

Pergunta

Qual seria a expressão da derivada de $\dfrac{\partial \mathcal{L}}{\partial w_1}$ no caso em que:

$$z_1 = w_1 x \qquad h_1 = e^{z_1} \qquad \hat{y} = 2h_1 \qquad \mathcal{L} = \hat{y}^2$$

Faça a solução por substituição direta e depois usando a regra da cadeia.
Resposta: $8xe^{2xw_1}$

3. Exemplo: Encontrando o parâmetro ótimo para uma Loss simples

Para visualizar a atualização e a otimização de um parâmetro sem a complexidade de uma rede inteira, considere que para uma entrada $x = 1$ a saída é $y = 3$.

Vamos adotar ainda que nosso viés b é 0, e que não iremos fazer nenhuma ativação, logo o único parâmetro que devemos otimizar é w, visto que:

$$\hat{y} = wx + b \Longrightarrow \hat{y} = wx$$

Dessa maneira temos então:

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(wx - y)^2$$

como para x = 1, sabemos que y = 3, ficamos com

$$\mathcal{L}(w) = \frac{1}{2}(w - 3)^2.$$

Agora queremos encontrar o valor de w que minimize nossa perda; como estamos em um caso trivial, é evidente que tal valor é $w = 3$; mas considere que a gente não saiba disso, e então vamos usar a estratégia de gradiente descendente pra chegar a tal valor. Lembrando então que

$$\boxed{w_{\text{novo}} = w_{\text{antigo}} - \eta \frac{\partial \mathcal{L}}{\partial w}}.$$

então a derivada que calculamos é

$$\frac{d\mathcal{L}}{dw} = w - 3.$$

se adotarmos a taxa de aprendizado como $\eta = 0{,}2$ e o valor inicial de $w_0 = 0$ (não recomendado para problemas realísticos e complexos), a regra de atualização fica

$$w_{k+1} = w_k - 0{,}2(w_k - 3).$$

Algumas iterações são apresentadas na tabela a seguir

k	w_k	$\mathcal{L}(w_k)$
0	0,000000	4,50000000
1	0,600000	2,88000000
2	1,080000	1,84320000
3	1,464000	1,17964800
5	2,016960	0,48318382
10	2,677877	0,05188147
15	2,894447	0,00557073
20	2,965412	0,00059815
25	2,988666	0,00006423
29	2,995358	0,00001078

visualmente, o que estamos fazendo é "andar" pela função de perda até chegar ao valor mínimo, como representado na figura abaixo.

O algoritmo não precisa conhecer antecipadamente o valor 3. Ele apenas observa a inclinação atual e dá pequenos passos na direção de menor perda. Em uma rede neural, a mesma lógica é aplicada simultaneamente a muitos parâmetros.

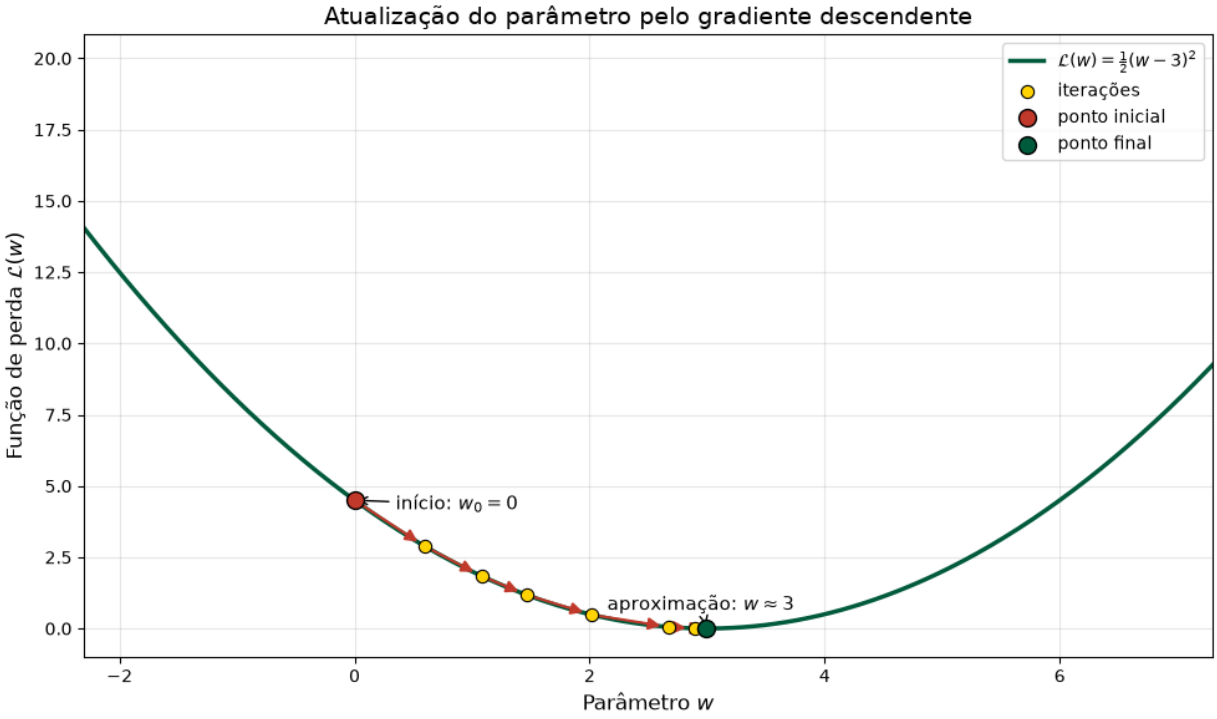


Figure 1: Representação visual para algumas iterações

4. Sobre o conceito de diferenciação automática

Quando trabalhamos com redes neurais realistas, precisamos encontrar as derivadas associadas a centenas, milhares, milhões de parâmetros. A diferenciação automática é a estratégia que aplica a regra da cadeia às operações da rede. Quando estamos trabalhando com PINNs ela aparece em dois lugares diferentes:

Objetivo	**Derivadas calculadas**
Treinar a rede	$\partial\mathcal{L}/\partial\theta_i$ em relação aos pesos e vieses
Avaliar a física	$\partial u_{\boldsymbol{\theta}}/\partial t$, $\partial u_{\boldsymbol{\theta}}/\partial x$ e $\partial^2 u_{\boldsymbol{\theta}}/\partial x^2$

A ideia essencial é: a rede produz uma saída, a perda mede o quanto essa saída está inadequada, o gradiente informa como modificar os parâmetros e o otimizador repete o processo.

5. Como a física entra na função de perda

Considere uma equação diferencial escrita de forma genérica:

$$\mathcal{F}\left(x, u, \frac{\partial u}{\partial x}, \frac{\partial^2 u}{\partial x^2}, \dots\right) = 0.$$

A rede aproxima a solução por $u(x) \approx u_{\boldsymbol{\theta}}(x)$. Substituindo a rede na equação, obtemos o resíduo:

$$r_{\boldsymbol{\theta}}(x) = \mathcal{F}\left(x, u_{\boldsymbol{\theta}}, \frac{\partial u_{\boldsymbol{\theta}}}{\partial x}, \frac{\partial^2 u_{\boldsymbol{\theta}}}{\partial x^2}, \dots\right).$$

A perda física é construída para que o resíduo seja pequeno nos pontos internos:

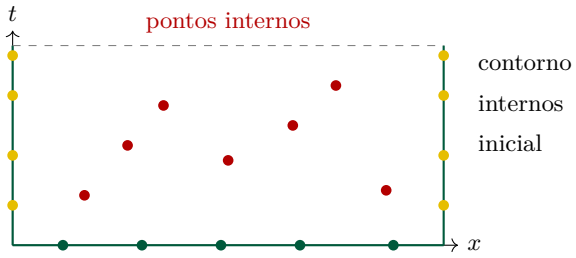
$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f}\sum_{j=1}^{N_f} r_{\boldsymbol{\theta}}(x_j)^2.$$

As condições de contorno e iniciais também entram na perda:

$$\mathcal{L}_{\text{PINN}} = \lambda_f \mathcal{L}_{\text{PDE}} + \lambda_b \mathcal{L}_{\text{BC}} + \lambda_i \mathcal{L}_{\text{IC}}.$$

6. Pontos de colocação e exemplo térmico

Os pontos utilizados não formam necessariamente uma malha tradicional. Eles são amostras do domínio onde a rede deve satisfazer a equação e as condições.



Para a condução de calor unidimensional, a rede recebe (x, t) e produz $u_{\boldsymbol{\theta}}(x, t)$:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \qquad 0 < x < L, \quad 0 < t < 1,$$

com

$$u(0, t) = u(L, t) = 0, \qquad u(x, 0) = \sin\left(\frac{\pi x}{L}\right).$$

O resíduo da EDP utilizado pela PINN é expresso por:

$$r_{\boldsymbol{\theta}}(x, t) = \frac{\partial u_{\boldsymbol{\theta}}}{\partial t} - \alpha \frac{\partial^2 u_{\boldsymbol{\theta}}}{\partial x^2}.$$

7. Correspondência com o DeepXDE

A implementação pode ser lida como uma tradução direta da formulação matemática:

Formulação	**DeepXDE**
Domínio espacial e temporal	`Interval`, `TimeDomain`, `GeometryXTime`
Resíduo da equação	função `pde(x, y)`
Condição de contorno	`DirichletBC`
Condição inicial	`IC`
Pontos de treinamento	`TimePDE`
Rede $u_{\boldsymbol{\theta}}(x, t)$	`dde.nn.FNN`
Otimização	`model.compile` e `model.train`
Validação	`model.predict` e solução analítica

Para um problema direto, a solução analítica não precisa ser conhecida durante o treinamento. Ela pode ser usada depois apenas para validar a solução da PINN.

Mensagem final para a turma

Uma PINN não substitui a matemática do problema. Ela usa uma rede neural como aproximadora e transforma a equação diferencial, as condições e os dados disponíveis em uma função de perda. O treinamento repete o ciclo:

rede neural como função + perda física + condições do problema + otimização = PINN